47

BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

Structured Concurrency: StructuredTaskScope ile Kaotik Thread'leri Hizaya Sokma

Structured Concurrency: StructuredTaskScope (ShutdownOnFailure & Success)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                UNSTRUCTURED VS STRUCTURED CONCURRENCY        │
└──────────────────────────────────────────────────────────────┘

1. KAOTİK (UNSTRUCTURED):
   Ana Görev ──┬──► Alt Görev 1 (Uçak Bileti) ──► [ HATA PATLADI! 💥 ]
               └──► Alt Görev 2 (Otel Rezerv.) ──► [ YETİM KALDI! (Sonsuza dek çalışır!) ]
   ⚠ Ana metod dönse bile Alt Görev 2 arka planda para ve CPU harcamaya devam eder!

2. YAPISAL (STRUCTURED - StructuredTaskScope):
   try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
       scope.fork(altGorev1); // Sanal thread'de çalışır
       scope.fork(altGorev2); // Sanal thread'de çalışır
       scope.join();          // Birlikte başlarlar, birlikte biterler!
       scope.throwIfFailed(); // Biri hata verirse DİĞERLERİ OTOMATİK İPTAL EDİLİR!
   } // Scope kapanırken yetim thread kalması İMKÂNSIZDIR!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski asenkron kod, 3 çocuğunu lunaparkın ortasında serbest bırakıp eve dönen bir ebeveyne benzer. Biri kaybolsa diğerleri akşama kadar dönme dolapta boşuna para harcar (yetim thread). Structured Concurrency ise çocukların ellerini tutarak parka girmektir. Hep birlikte girersiniz, hep birlikte çıkarsınız. Birinin ayağı takılıp düşerse diğer ikisini de hemen durdurup eve dönersiniz; kimse kaybolmaz, para israf olmaz!

🇬🇧 ENGLISH (ELI5)

Unstructured concurrency is dropping three kids off at an amusement park and driving home. If one breaks an arm, the other two keep wandering and riding rollercoasters indefinitely (orphaned threads). Structured Concurrency is holding their hands together in a single group. You enter together and leave together. If one gets injured, you immediately cancel the outing for everyone. No thread is ever abandoned in the wild!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

StructuredTaskScope ve Yaşam Döngüsü Yönetimi: 1) Yapısal Programlama Disiplini: Tıpkı `if-else` veya `for` döngülerinin tek bir giriş ve çıkış noktası olması gibi, eşzamanlı görevlerin de aynı blokta başlayıp aynı blokta sonlanması kuralıdır (Lexical Scoping). 2) Kısa Devre İptali (Short-Circuit Cancellation): `ShutdownOnFailure` politikasında alt görevlerden biri başarısız olursa, kapsamdaki diğer tüm kardeş görevler otomatik olarak `interrupt` edilir; boşuna ağ ve CPU tüketilmez. 3) İlk Kazanan Politikası (`ShutdownOnSuccess`): Dağıtık bir kütüphanede aynı veriyi 3 farklı yedekten ararken ilk geçerli yanıtı alan görev kazanır, diğer 2 sorgu anında iptal edilir. 4) Hata Ayıklama (Observability): Thread dökümlerinde (Thread Dump) hiyerarşik ebeveyn-çocuk ilişkisi net biçimde görünür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

StructuredTaskScope Mechanics: 1) Lexical Scoping for Concurrency: Paralleling structured control flows (`if/while`), concurrent tasks are bound to explicit lexical scopes. Subtasks spawned within a block are guaranteed to terminate before the block exits. 2) Eliminating Thread Leakage: In unstructured models (`CompletableFuture.supplyAsync`), failed tasks leave sibling tasks running as invisible orphans. `StructuredTaskScope.ShutdownOnFailure` automatically intercepts exceptions, issuing immediate cancellation interrupts to surviving sibling threads. 3) Hedged Optimization via `ShutdownOnSuccess`: In multi-replica queries, the first subtask to return resolves the scope, aborting remaining parallel requests. 4) Observability: Thread dumps reflect pristine hierarchical parent-child trees rather than flat unorganized lists.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: StructuredTaskScope.ShutdownOnFailure ile Güvenli Tatil Rezervasyonu
import java.util.concurrent.StructuredTaskScope;
import java.util.function.Supplier;

public class StructuredReservationService {
    record Flight(String flightNo, double price) {}
    record Hotel(String hotelName, double price) {}
    record VacationPackage(Flight flight, Hotel hotel) {}

    static Flight bookFlight() throws InterruptedException {
        Thread.sleep(100);
        return new Flight("TK-1923", 450.0);
    }

    static Hotel bookHotel() throws InterruptedException {
        Thread.sleep(80);
        return new Hotel("Grand Hyatt", 600.0);
    }

    public VacationPackage bookVacation() throws Exception {
        // Alt görevlerden biri patlarsa diğerini otomatik iptal et!
        try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
            // İki bağımsız görevi sanal thread'ler üzerinde çatalla (fork)
            Supplier<Flight> flightSubtask = scope.fork(StructuredReservationService::bookFlight);
            Supplier<Hotel> hotelSubtask = scope.fork(StructuredReservationService::bookHotel);

            scope.join();           // İki görevin bitmesini bekle
            scope.throwIfFailed();  // Biri hata verdiyse burada fırlat!

            // Her ikisi de başarılı oldu; sonuçları birleştir
            return new VacationPackage(flightSubtask.get(), hotelSubtask.get());
        } // Scope çıkışında hiçbir thread açık kalamaz!
    }

    void main() throws Exception {
        var service = new StructuredReservationService();
        VacationPackage trip = service.bookVacation();
        System.out.println("Rezervasyon Başarılı: " + trip);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

StructuredTaskScope try-with-resources bloğunda join() çağırmayı unutursanız IllegalStateException fırlatılır! Java çalışma zamanı, tüm alt thread'lerin sonucunu beklemeden scope'tan çıkmanıza ASLA izin vermez.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Attempting to retrieve subtask results via `.get()` prior to calling `scope.join()` throws an `IllegalStateException`. StructuredTaskScope strictly enforces that scopes cannot exit until all child threads have settled.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

• StructuredTaskScope ile alt görevler tek bir blokta başlar ve biter. Biri patlarsa kardeş görevler otomatik iptal edilir, yetim thread kalmaz.

• StructuredTaskScope enforces lexical task lifecycles: tasks begin and end together. Failure in one sibling immediately cancels the rest.